TapToGo: A Comprehensive Project Analysis

1. Abstract & Introduction

TapToGo is a robust, role-based smart transit management application designed to handle ticketless bus boarding, live occupancy tracking via Artificial Intelligence, and real-time location monitoring. The project utilizes Python with the **Flet** framework for cross-platform UI rendering, SQLite for local data persistence, and a PyTorch-based Deep Learning model (**CSRNet**) for automated passenger counting. It caters to four primary personas: Admin, Passenger, Conductor, and Camera Device, creating a seamless, end-to-end digitized transit experience.

2. System Architecture

The application follows a monolith, local-first architecture built efficiently:

- **Frontend UI Layer:** Built using Flet (`main.py` , `screens/*` , `components/*`), enabling a reactive, component-guided UI that compiles seamlessly to mobile (Android) and desktop environments from a single Python codebase.
- **Database Layer:** A local SQLite database (`db.py`) handles all application state—including user wallets, bus schedules, tap logs, and conductor credentials.
- **AI Engine:** A pre-trained CSRNet model (`model_function.py` , `csrnet.pth`) powered by PyTorch processes optical data (images, videos, or webcam streams) to estimate crowd density on the bus.
- **Hardware Integration Layer:**
  - **NFC:** Bridged via a background file polling mechanism (`nfc_scan.tmp`), allowing native OS APIs (like Android Kotlin) to pass NFC UID scans to the Python backend.
  - **GPS & Mapping:** Utilizes the Google Maps Static API for rendering live bus locations during transit.

3. Role-Based Workflows

Admin

The Admin provisions the entire ecosystem. They are responsible for adding bus metadata, defining route stops and schedules, assigning conductors to specific buses, generating temporary camera device credentials, and mapping physical NFC tag signatures to their digital bus counterparts. Once set up, the admin can edit or manage the fleet dynamically.

Passenger

1. **Registration & Wallet:** Passengers register an account and can top up an enclosed digital wallet dynamically without linking to external bank accounts.
2. **Discovery & Boarding:** Users search for buses between stops. Upon boarding, they tap their phone against a physical NFC sticker mounted near the bus door. The app detects the NFC tag, identifies the bus, and prompts the user to select their "Tap-in" and "Destination" stops.
3. **In-Transit:** Once boarding is confirmed, the "On Board" screen activates. It displays the real-time bus location (plotted via Google Maps Static API) and current passive occupancy count.
4. **Exit & Payment:** Upon alighting, the user taps "Exit Bus". The system safely calculates the fare internally based on the number of stops traversed (e.g., ₹5 per stop, minimum ₹5). A strict state-lock forces the application into a payment screen until the calculated fare is settled from their wallet or via UPI.

Conductor

Conductors log into a dedicated dashboard tied explicitly to their assigned bus. They can:

- Manually trigger the AI-based passenger counting system by uploading live photos or videos from the bus interior.
- View a real-time ledger of all passenger NFC tap-ins and tap-outs.
- Broadcast the bus's current GPS location to the central database utilizing the device's native GPS, thereby updating the live map for all onboard passengers.

Camera Device (Automated Counting)

A dedicated, headless role intended for a spare smartphone mounted inside the bus. Logging into the "Camera Portal" automatically spins up a background thread that captures the live camera feed, pipes frames through the CSRNet PyTorch model every 10 seconds, and writes min/max occupancy thresholds directly to the database. This allows passengers and conductors to see live bus crowdedness completely passively.

4. Technical Implementation Details

Database Schema (`db.py`)

- `users` : Stores passenger authentication, wallet balances, and active trip states (e.g., `payment_pending` , `current_bus_id`).
- `buses` : Holds bus metadata, assigned credentials, current geo-coordinates (`current_lat` , `current_lng`), and AI-predicted `people_count` .
- `conductors` & `cameras` : Authentication tables linking temporary credentials to specific `bus_id` s.
- `tap_logs` : An immutable ledger of all boarding and exiting actions used heavily for system audits and fair fare calculation.

AI Crowd Counting (`model_function.py`)

TapToGo leverages a **CSRNet** (Congested Scene Recognition Network) implemented natively in PyTorch. The structure uses the first 23 layers of a robust VGG-16 network as a feature extractor (frontend). This is followed by a configuration of dilated convolutional layers (backend) designed to maintain spatial resolution while expanding the receptive field. The ultimate output is a heatmap density matrix, the sum of which yields the estimated crowd count. Batch processing is handled elegantly in a background thread to prevent UI freezing.

NFC Bridging (`screens/tap.py`)

Because Flet currently lacks native cross-platform NFC module hooks, the system applies an innovative asynchronous file-based bridge. A background `asyncio` loop continuously checks for a temporary file `nfc_scan.tmp` across logical directory trees. When an external native Android service detects an NFC tag, it writes the UID string into this file. The Python application detects changes, clears the file, and proceeds with the boarding logic safely.

5. Conclusion

TapToGo presents a deeply integrated, highly pragmatic entry into smart transit solutions. By combining offline-first database principles, seamless cross-platform Flet UI wrappers, resourceful hardware bridging for NFC interfaces, and localized state-of-the-art deep learning for passive crowd monitoring, it offers a highly scalable and robust architecture suitable for modernizing both municipal and private transport fleets.